

Graph 1 - Adjacency and Traversals

All edges are undirected. Start vertex: 1. DFS and BFS visit neighbors in ascending numeric order. Edge weights do not affect traversal order.

1. Adjacency Matrix

Weighted matrix: 0 denotes no edge (and the diagonal). For a binary adjacency matrix, replace every nonzero entry by 1. Row and column labels follow the same order.

Vertex	0	1	2	3	4	5
0	0	6	1	5	0	0
1	6	0	5	0	3	0
2	1	5	0	5	6	4
3	5	0	5	0	0	2
4	0	3	6	0	0	5
5	0	0	4	2	5	0

2. Adjacency List

0: 1(6), 2(1), 3(5)
1: 0(6), 2(5), 4(3)
2: 0(1), 1(5), 3(5), 4(6), 5(4)
3: 0(5), 2(5), 5(2)
4: 1(3), 2(6), 5(5)
5: 2(4), 3(2), 4(5)

Notation: neighbor(weight).

3. Depth-First Search (DFS)

Visit order: 1 -> 0 -> 2 -> 3 -> 5 -> 4

DFS tree edges: 1-0, 0-2, 2-3, 3-5, 5-4

Steps: Visit 1; Visit 0; Visit 2; Visit 3; Visit 5; Visit 4; Backtrack 4 to 5; Backtrack 5 to 3; Backtrack 3 to 2; Backtrack 2 to 0; Backtrack 0 to 1.

Graph 1 - Breadth-First Search

4. Breadth-First Search (BFS)

Visit order: 1 -> 0 -> 2 -> 4 -> 3 -> 5

BFS tree edges: 1-0, 1-2, 1-4, 0-3, 2-5

Process	Newly discovered	Queue after processing
1	0, 2, 4	0, 2, 4
0	3	2, 4, 3
2	5	4, 3, 5
4	-	3, 5
3	-	5
5	-	empty

Traversal order records first visits; tree edges show the actual discovery links. Consecutive vertices in a traversal order need not be adjacent.

Graph 1 - MST and Shortest Paths

5. Minimum Spanning Tree - Prim's Algorithm

Start with $S = \{1\}$. At every step, choose the minimum-weight edge joining S to a vertex outside S . Break ties by the numeric pair of edge endpoints.

Step	Chosen edge	Weight	Vertices in S	Total
1	1-4	3	1, 4	3
2	1-2	5	1, 2, 4	8
3	2-0	1	0, 1, 2, 4	9
4	2-5	4	0, 1, 2, 4, 5	13
5	5-3	2	0, 1, 2, 3, 4, 5	15

MST edges: 1-4 (3), 1-2 (5), 2-0 (1), 2-5 (4), 5-3 (2).

Total MST weight = 15.

6. Shortest Paths - Dijkstra's Algorithm

Initialize $d(1) = 0$ and all other distances to infinity. Settle the unvisited vertex with smallest tentative distance, then relax its incident edges. Ties choose the lower vertex number; an equal-distance relaxation keeps the existing predecessor.

Settle	d(0)	d(1)	d(2)	d(3)	d(4)	d(5)
1	6	0	5	inf	3	inf
4	6	0	5	inf	3	8
2	6	0	5	10	3	8
0	6	0	5	10	3	8
5	6	0	5	10	3	8
3	6	0	5	10	3	8

Destination	Shortest path from 1	Distance
0	1 -> 0	6
1	1	0
2	1 -> 2	5
3	1 -> 2 -> 3	10
4	1 -> 4	3
5	1 -> 4 -> 5	8

Graph 2 - Adjacency and Traversals

All edges are undirected. Start vertex: 1. DFS and BFS visit neighbors in ascending numeric order. Edge weights do not affect traversal order.

1. Adjacency Matrix

Weighted matrix: 0 denotes no edge (and the diagonal). For a binary adjacency matrix, replace every nonzero entry by 1. Row and column labels follow the same order.

Vertex	1	2	3	4	5	6	7	8	9
1	0	10	9	6	12	0	0	0	0
2	10	0	0	0	8	0	0	0	0
3	9	0	0	7	0	5	0	0	0
4	6	0	7	0	8	8	7	0	0
5	12	8	0	8	0	0	4	0	13
6	0	0	5	8	0	0	14	6	0
7	0	0	0	7	4	14	0	8	8
8	0	0	0	0	0	6	8	0	10
9	0	0	0	0	13	0	8	10	0

2. Adjacency List

1: 2(10), 3(9), 4(6), 5(12)
2: 1(10), 5(8)
3: 1(9), 4(7), 6(5)
4: 1(6), 3(7), 5(8), 6(8), 7(7)
5: 1(12), 2(8), 4(8), 7(4), 9(13)
6: 3(5), 4(8), 7(14), 8(6)
7: 4(7), 5(4), 6(14), 8(8), 9(8)
8: 6(6), 7(8), 9(10)
9: 5(13), 7(8), 8(10)

Notation: neighbor(weight).

3. Depth-First Search (DFS)

Visit order: 1 -> 2 -> 5 -> 4 -> 3 -> 6 -> 7 -> 8 -> 9

DFS tree edges: 1-2, 2-5, 5-4, 4-3, 3-6, 6-7, 7-8, 8-9

Steps: Visit 1; Visit 2; Visit 5; Visit 4; Visit 3; Visit 6; Visit 7; Visit 8; Visit 9; Backtrack 9 to 8; Backtrack 8 to 7; Backtrack 7 to 6; Backtrack 6 to 3; Backtrack 3 to 4; Backtrack 4 to 5; Backtrack 5 to 2; Backtrack 2 to 1.

Graph 2 - Breadth-First Search

4. Breadth-First Search (BFS)

Visit order: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 9 -> 8

BFS tree edges: 1-2, 1-3, 1-4, 1-5, 3-6, 4-7, 5-9, 6-8

Process	Newly discovered	Queue after processing
1	2, 3, 4, 5	2, 3, 4, 5
2	-	3, 4, 5
3	6	4, 5, 6
4	7	5, 6, 7
5	9	6, 7, 9
6	8	7, 9, 8
7	-	9, 8
9	-	8
8	-	empty

Traversal order records first visits; tree edges show the actual discovery links. Consecutive vertices in a traversal order need not be adjacent.

Graph 2 - MST and Shortest Paths

5. Minimum Spanning Tree - Prim's Algorithm

Start with $S = \{1\}$. At every step, choose the minimum-weight edge joining S to a vertex outside S . Break ties by the numeric pair of edge endpoints.

Step	Chosen edge	Weight	Vertices in S	Total
1	1-4	6	1, 4	6
2	4-3	7	1, 3, 4	13
3	3-6	5	1, 3, 4, 6	18
4	6-8	6	1, 3, 4, 6, 8	24
5	4-7	7	1, 3, 4, 6, 7, 8	31
6	7-5	4	1, 3, 4, 5, 6, 7, 8	35
7	5-2	8	1, 2, 3, 4, 5, 6, 7, 8	43
8	7-9	8	1, 2, 3, 4, 5, 6, 7, 8, 9	51

MST edges: 1-4 (6), 4-3 (7), 3-6 (5), 6-8 (6), 4-7 (7), 7-5 (4), 5-2 (8), 7-9 (8).

Total MST weight = 51.

6. Shortest Paths - Dijkstra's Algorithm

Initialize $d(1) = 0$ and all other distances to infinity. Settle the unvisited vertex with smallest tentative distance, then relax its incident edges. Ties choose the lower vertex number; an equal-distance relaxation keeps the existing predecessor.

Settle	d(1)	d(2)	d(3)	d(4)	d(5)	d(6)	d(7)	d(8)	d(9)
1	0	10	9	6	12	inf	inf	inf	inf
4	0	10	9	6	12	14	13	inf	inf
3	0	10	9	6	12	14	13	inf	inf
2	0	10	9	6	12	14	13	inf	inf
5	0	10	9	6	12	14	13	inf	25
7	0	10	9	6	12	14	13	21	21
6	0	10	9	6	12	14	13	20	21
8	0	10	9	6	12	14	13	20	21
9	0	10	9	6	12	14	13	20	21

Destination	Shortest path from 1	Distance
1	1	0
2	1 -> 2	10
3	1 -> 3	9
4	1 -> 4	6
5	1 -> 5	12
6	1 -> 4 -> 6	14
7	1 -> 4 -> 7	13
8	1 -> 4 -> 6 -> 8	20
9	1 -> 4 -> 7 -> 9	21